

Session-Based Recommender Systems

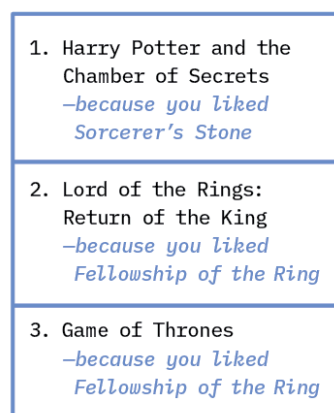
☰ Journal/Conference	Research report/ Blog
⚙ Status	Done

Session-Based Recommender Systems

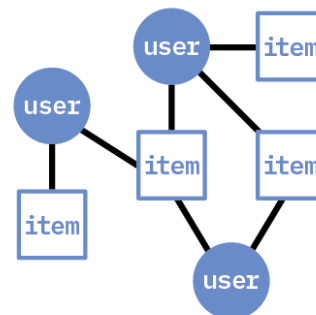
Hệ gợi ý theo phiên

Bài toán này không mới và đạt nhiều thành tựu trong 10 năm qua với các cách tiếp cận khác nhau.

Các hệ gợi ý cổ điển gồm: Lọc dựa trên nội dung, lọc cộng tác hoặc các hướng tiếp cận kết hợp khía cạnh của 2 thuật toán này.



Content-based



Collaborative Filtering

Giả định cơ bản của cả hai hệ thống là tất cả các tương tác lịch sử đều quan trọng như nhau đối với sở thích hiện tại của người dùng. Trong thực tế có thể không đúng.

Việc lựa chọn còn phụ thuộc vào sở thích ngắn hạn và gần đây hơn.

Ví dụ: Các mặt hàng "đã xem/mua gần đây" có thể phù hợp hơn những mặt hàng khác dù các mặt hàng này có thể chiếm tỷ lệ nhỏ trong các tương tác

quá khứ.

Sở thích người dùng đối với một danh mục nhất định có xu hướng thay đổi theo thời gian nên việc xét toàn bộ các tương tác trước đây thay vì ưu tiên các danh mục gần đây có thể không hợp lý.

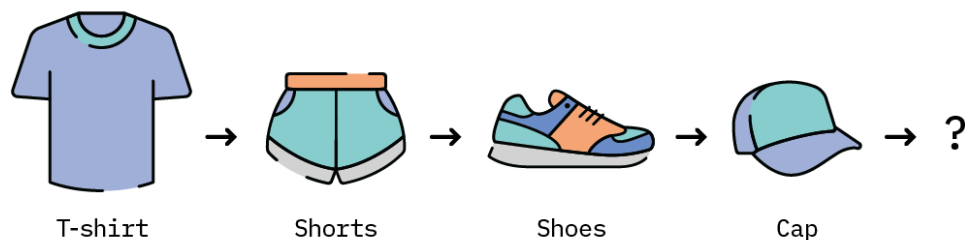
→ Thuật toán gợi ý dựa trên phiên: Phụ thuộc vào các tương tác gần đây của người dùng thay vì sử dụng sở thích lịch sử của người dùng.

Đặc điểm: người dùng có thể xuất hiện ẩn danh, không cần đăng nhập hoặc duyệt web ẩn danh mà vẫn được hệ thống gợi ý sản phẩm phù hợp.

Tại sao lại phổ biến?

- Có thể triển khai ngay cả khi không có historical user data và không phụ thuộc vào phân phối ng dùng
- Có nhiều Dataset public tập trung vào phiên được phát hành tạo điều kiện cho nghiên cứu trong lĩnh vực này
- Sự phát triển của các phương pháp DL (trong sequential data) kéo theo phát triển của hệ khuyến nghị dựa trên phiên

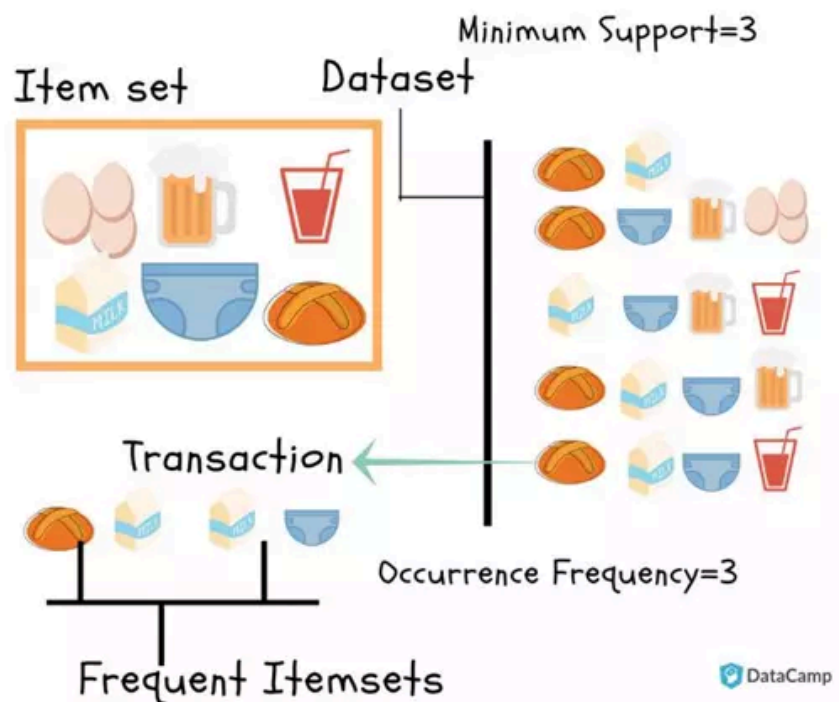
Bài toán Session based recommender



- Session: Là tập hợp nhiều tương tác của người dùng diễn ra cùng nhau trong khoảng thời gian liên tiếp.
 - Session có thể diễn ra trong cùng ngày/ vài ngày/ vài tuần/ vài tháng.
- Mục tiêu là dự đoán sản phẩm trong phiên mà người dùng “đủ” ưng để click. Tối đa khả năng người dùng click vào ít nhất 1 trong số sản phẩm mà hệ thống đề xuất.
 - Gọi là next event prediction (NEP): Cho 1 chuỗi sự kiện và dự đoán sự kiện tiếp theo.

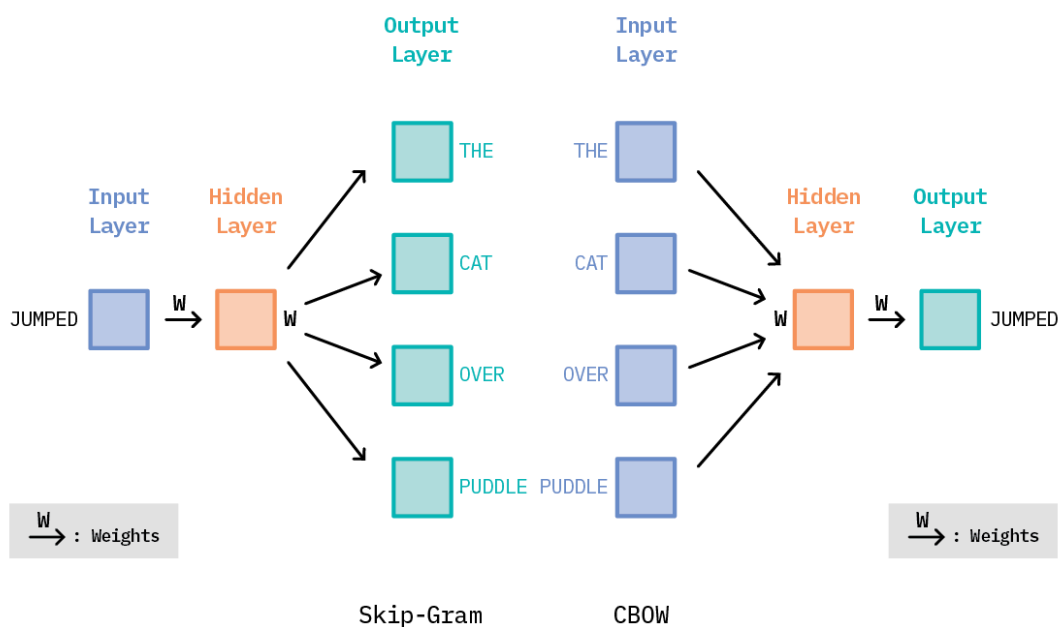
Mô hình hóa hệ gợi ý theo phiên

- Có nhiều baseline cho bài toán NEP
 - Đơn giản và phổ biến nhất là sử dụng tần suất đồng xuất hiện với item cuối của phiên.
 - Còn gọi là luật kết hợp (Association rule). Cách heuristic này khá rõ ràng, tuy nhiên không nắm bắt được hết độ phức tạp trong lịch sử tương tác của người dùng.



- Gần đây, các hướng tiếp cận dựa trên DL đẩy lên những làn sóng mới. Các biến thể của GNN và RNN được áp dụng cho bài toán này với kết quả đầy hứa hẹn, SOTA cho NEP trong một số UC.
 - Nắm bắt được sự phức tạp tương tác trong phiên, nhưng có thể khó hiểu, không trực quan, không phải luôn tốt hơn thuật toán đơn giản (khi xét về accuracy)
- Ngoài ra còn có option khác là word2vec: Nắm giữa heuristic và DL. Là mô hình nắm bắt được ngữ nghĩa chỉ với 1 lớp.

Huấn luyện như bài toán NLP



Ta có thể coi:

- Mỗi phiên như một câu, trong đó mỗi item trong phiên như một từ.
- Tập lịch sử trình duyệt trên website của người dùng là corpus

Word2vec sẽ xử lý toàn bộ kho ngữ liệu, học quan hệ giữa các item trong ngữ cảnh của phiên. Kết quả sẽ là tập embedding, mỗi một cái đại diện cho mỗi item.

Ý tưởng là các embedding của item học được sẽ chứa nhiều thông tin hơn cách heuristic đơn giản và huấn luyện word2vec nhanh và đơn giản hơn huấn luyện các mô hình phức tạp và cần nhiều dữ liệu như DL.

→ Hợp lý khi bài toán này là tuần tự một cách tự nhiên và thứ tự của chúng chứa thông tin.

Word2vec applied to Recommendation: Hyperparameters Matter.

Tuning Word2vec for Large Scale Recommendation Systems

Evaluating session-based recommenders

Đánh giá gồm 2 giai đoạn: offline và online.

- Offline: Lịch sử người dùng được coi là gold standard. Hiệu quả thuật toán được đo lường bằng khả năng dự đoán sản phẩm trong phiên. Có nhiều cách
 - Giữ lại phần tử cuối của mỗi phiên.
 - Che đi một phần dữ liệu trong phiên rồi kiểm tra xem thuật toán có dự đoán đúng phần bị che không.
 - Nếu người dùng có nhiều phiên, giữ lại toàn bộ phiên cuối.
- Online: Đánh giá trong môi trường thật khi hệ thống đang phục vụ người dùng thật.
 - A/B testing, conversion, thời gian xem...
 - Mục tiêu là xem hệ gợi ý có thực sự cải thiện hành vi người dùng không.

Evaluation metrics

Với một chuỗi gồm n tương tác, ta dùng $n-1$ item đầu tiên của chuỗi làm training set. Sau đó ta lấy mẫu ngẫu nhiên các cặp (x_{n-1}, x_n) từ các chuỗi cho tập val và tập test

Khi dự đoán, ta dùng phần tử cuối cùng trong chuỗi huấn luyện x_{n-1} là item truy vấn, sau đó dự đoán K item gần nhất với item truy vấn = độ tương đồng cosine giữa các biểu diễn vector.

Một vài metric để đánh giá: Recall@K, MRR@K

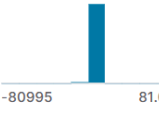
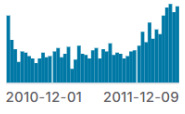
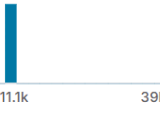
Experiments

Thực nghiệm trên dữ liệu Online retail:

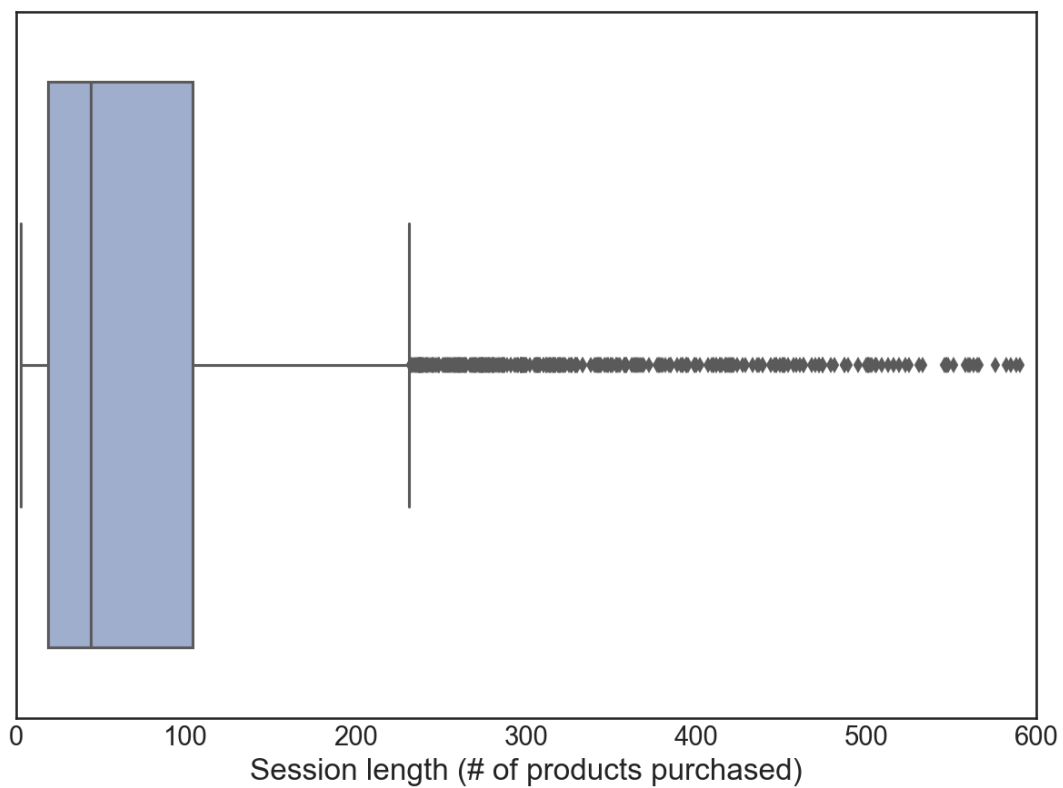
<https://www.kaggle.com/vijayuv/onlineretail>

Thu thập vào 12/1/2010 - 12/9/2011, chứa lịch sử thanh toán của 4372 khách và 3684 item (unique).

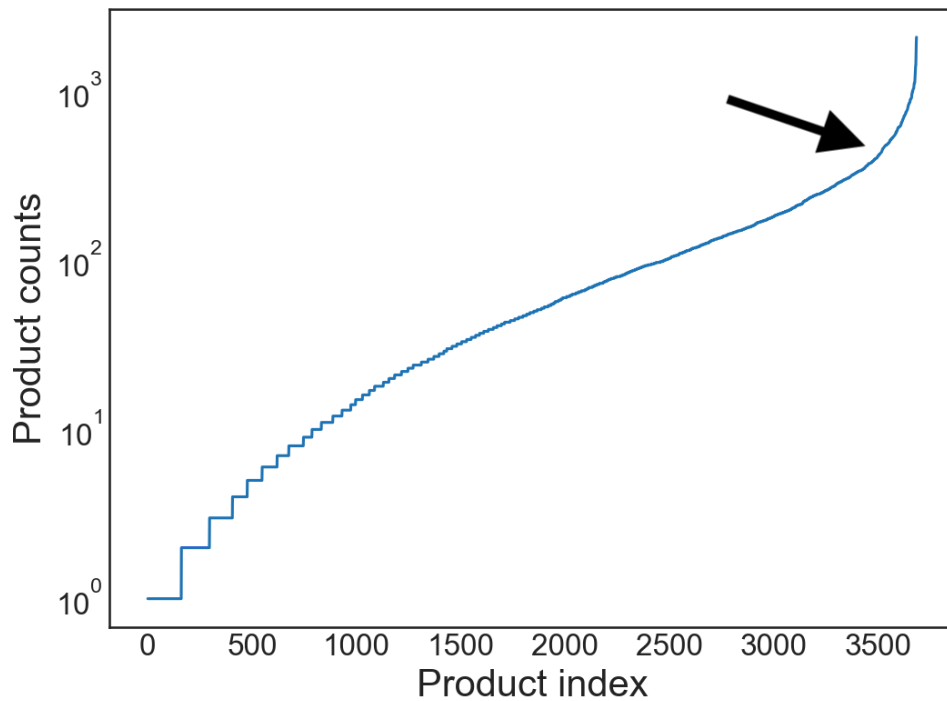
Dữ liệu này không chứa thứ tự sản phẩm được click mà chỉ có item chứa trong một khoảng thời gian. Vậy ta coi một phiên là toàn bộ lịch sử thanh toán của người dùng, do người dùng là unique.

StockCode	Description	# Quantity	InvoiceDate	# UnitPrice	Customer
4070 unique values	4224 unique values				
i5123A	WHITE HANGING HEART T-LIGHT HOLDER	6	12/1/2010 8:26	2.55	17850
i1053	WHITE METAL LANTERN	6	12/1/2010 8:26	3.39	17850
i4406B	CREAM CUPID HEARTS COAT HANGER	8	12/1/2010 8:26	2.75	17850
i4029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	12/1/2010 8:26	3.39	17850
i4029E	RED WOOLLY HOTTIE WHITE HEART.	6	12/1/2010 8:26	3.39	17850
i2752	SET 7 BABUSHKA NESTING BOXES	2	12/1/2010 8:26	7.65	17850
i1730	GLASS STAR FROSTED T-LIGHT HOLDER	6	12/1/2010 8:26	4.25	17850

Thay vì dự đoán xem người dùng đã click gì tiếp theo, ta dự đoán xem người dùng đã mua gì tiếp theo.



Theo thống kê thì trung vị mỗi người thanh toán 44 item, và trung bình thanh toán 96 item.



Biểu đồ log thống kê tần suất thanh toán của sản phẩm.

Nhìn longtail. Hầu hết sản phẩm không phổ biến và được thanh toán với số lần rất ít, đếm trên đầu ngón tay. Nhưng có một vài sản phẩm phổ biến và được thanh toán hàng nghìn lần.



Ô màu xanh: Training set
Cap: Query Item
Water bottle: Ground truth item

Các bước xử lý lưu ý:

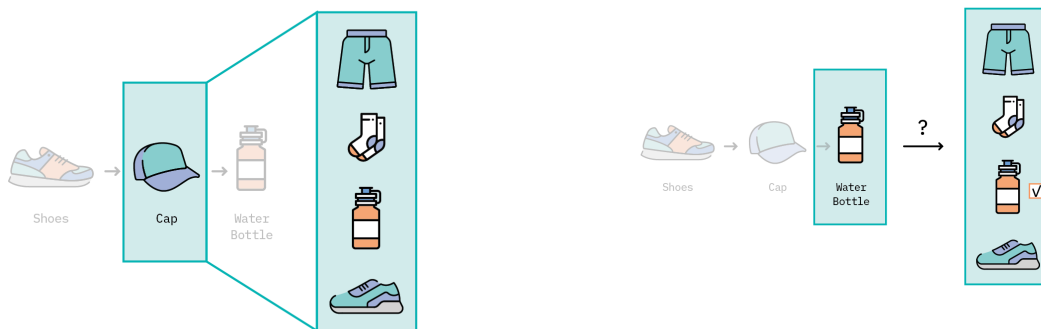
- Tập val và tập test không được có phần tử trùng nhau. Mỗi phần tử có dạng: (Query item, Ground truth item).
- Loại bỏ các session ít hơn 3 sản phẩm. Một phiên chỉ có 2 sản phẩm chỉ có 1 cặp (query, ground) và không cho chúng ta bất kỳ mẫu huấn luyện nào cả.

```

sessions = load_ecomm()
train, test, valid = train_test_split(sessions, test_size=1000)
model = Word2Vec(train, min_count=1, sg=1)
embeddings = model.wv
print(recall_at_k(test, embeddings, k=10))
print([mrr_at_k(test, embeddings, k=10)])

```

Tiếp theo chúng ta sinh ra gợi ý. Với một query item, ta sinh ra các gợi ý tương đồng nhất với item đó.



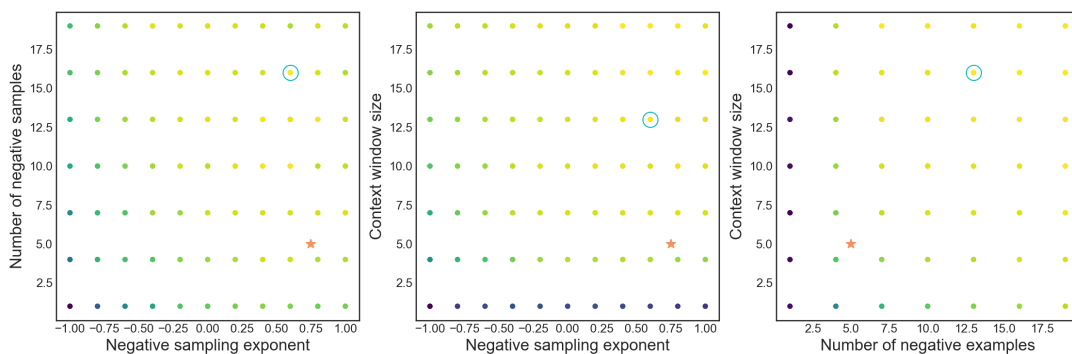
Với danh sách gợi ý thu được, ta chấm điểm mô hình bằng cách kiểm tra xem ground truth tương ứng có nằm trong danh sách gợi ý không.

We'll perform this set of operations for each [query item, ground truth item] pair in our test set, to compute an overall score using $Recall@K$ and $MRR@K$. The output of the code snippet above resulted in a $Recall@10$ of 19.7 and $MRR@10$ of 0.108. These results tell us that nearly 20% of the time, the user's true selection was included in the list of recommendations we generated.

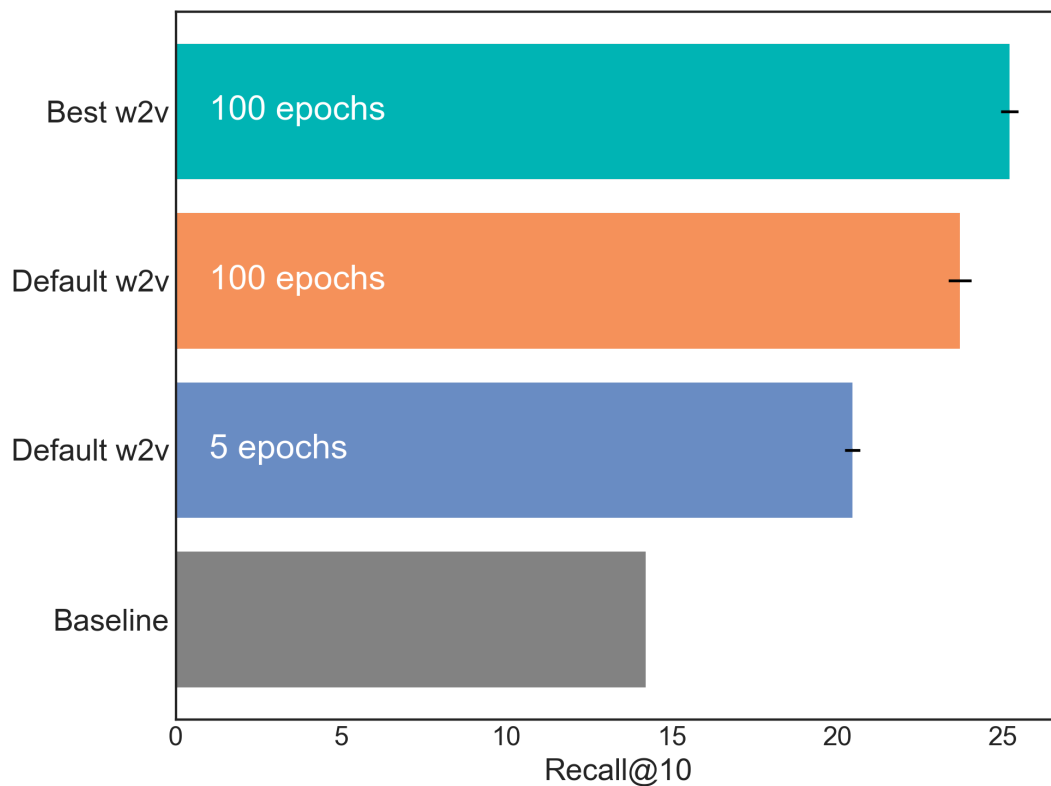
Hyperparameters matter

Trong phần trước, ta đơn giản là huấn luyện word2vec dùng siêu tham số mặc định. Ta có thể điều chỉnh hyperparameters.

- Context window size
 - Negative sampling exponent: Phân biệt cặp đúng và cặp nhiễu (Nền tảng của contrastive learning)
 - Mục tiêu huấn luyện Word2vec là tìm ra cách biểu diễn từ có ích cho việc dự đoán các từ xung quanh trong ngữ cảnh, thông qua tối ưu hóa trung bình logarit xác suất của tất cả các từ.
 - Xác suất này được mô hình hóa bằng hàm softmax. **Tính toán softmax** với kích thước từ điển như vậy thực sự **tốn kém** về tính toán. Tuy nhiên, ta có thể xấp xỉ softmax bằng kỹ thuật **“negative sampling”** - lấy mẫu âm. Mô hình phải cố gắng phân biệt một từ là thực sự trong context với những từ không trong context.
 - Ưu: Nhanh, đơn giản, nổi tiếng
- $$\log \sigma[s(x, y^+)] + \sum_{i=1}^K \log \sigma[-s(x, y_i^-)]$$
- Nhược: Không cho xác suất softmax đầy đủ, chất lượng phụ thuộc vào cách chọn negative samples. Phù hợp với học embedding hơn là calibrated probability.
- Number of negative samples



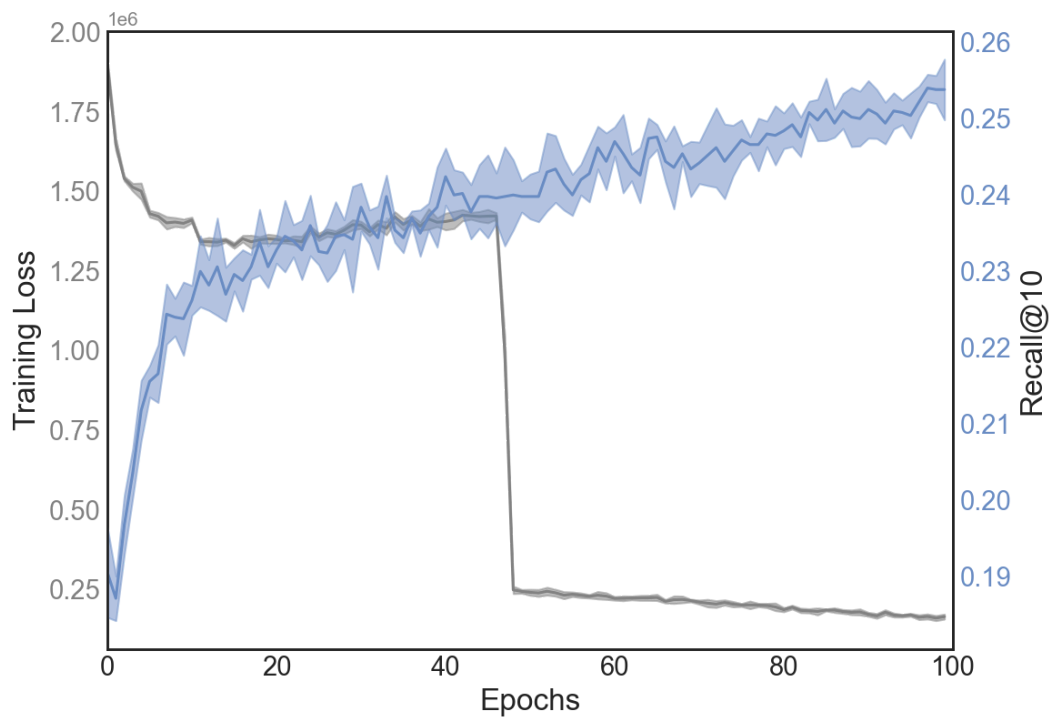
Results from our hyperparameter sweep: Each panel shows the Recall@10 scores (colored points, where yellow is a high score, and purple is a low score) associated with a unique configuration of hyperparameters. The best hyperparameter values for the Online Retail Data Set are denoted by the light blue circle. Word2vec’s default values are shown by the orange star. In all cases, the orange star is nowhere near the light blue circle, indicating that the default values are not optimal for this dataset.



Với NN truyền thống, khi train mô hình ta phải theo dõi training loss và validation loss. Mục tiêu là làm training loss giảm và validation loss giảm hoặc không tăng, và dừng khi có dấu hiệu overfitting.

Với word2vec, mục tiêu là học ra biểu diễn vector hữu ích cho downstream task chứ không phải loss của word2vec có giảm không. Do vậy train word2vec càng lâu càng tốt sẽ khiến embedding ổn định hơn, nắm bắt được nhiều cấu trúc đồng xuất hiện, cải thiện semantic similarity và tăng chất lượng downstream task. Train đến khi:

- Tài nguyên không cho phép
- Khi metric downstream plateau (ổn định)



Challenges

- **Computational Resources**
- **Metrics:** Recall@K và MRR@K là hai metric thường dùng cho NEP và cung cấp một metric chung để đánh giá các mô hình khác nhau. Tuy nhiên nó không thực sự tối ưu hóa mục tiêu của một doanh nghiệp như doanh thu, thời gian xem/ nghe...
 - Recall@K rất harsh. Nó chỉ kiểm tra item à người dùng thực sự chọn có nằm trong top K gợi ý không.
 - Phù hợp khi thứ tự trong danh sách không quá quan trọng (VD một trang web hiển thị 10-20 sản phẩm cùng lúc. Khi đó item đứng thứ 1 hay 8 không quan trọng. Quan trọng là item phải xuất hiện trong list trả về) → Recall@K phù hợp với các loại giao diện kiểu này.
 - Recall@K khắc nghiệt vì nó chấm đúng sai tuyệt đối, 0 hoặc 1 mà không cho điểm một phần (tức là điểm từ 0-1 thay vì fix cứng 0 hoặc 1).
 - MRR@K: Mean reciprocal rank, không chỉ xét item đúng có nằm trong top k không mà còn hỏi nếu có, nó nằm ở vị trí thứ mấy.

- $\text{Reciprocal Rank} = \frac{1}{\text{rank}}$
- Phù hợp khi thứ tự gợi ý là quan trọng.
- Cũng harsh chứ không chấm theo kiểu 1 phần.
- Các metric này chưa đủ trong thực tế vì nó không tương quan với các KPI như: Doanh thu tăng bao nhiêu, CTR tăng không, thời gian xem/nghe tăng không, tỷ lệ mua hàng tăng không, mức độ hài lòng của người dùng tăng không?...
- Nên online A/B testing để xem hệ thống có thực sự cải thiện hành vi người dùng ngoài đời không. Ví dụ: Nhóm A dùng resys cũ, nhóm B dùng resys mới. Sau đó so sánh về CTR, conversion, watch time, revenue...
- **Cold start:** Trong resys nói chung, cold start gồm hai loại: New users and new products.
 - Với session based, new user không phải vấn đề. Miễn là web có một số base của người dùng trước thì dữ liệu đó có thể được sử dụng để sinh embedding cho các sản phẩm hiện có dựa trên cách người dùng điều hướng trang web. Sau đó các embedding có thể tạo ra gợi ý phù hợp cho người dùng mới.
 - Item mới vẫn là thách thức → không có session đi kèm với chúng trong training set → không có embedding đi kèm với chúng. Một giải pháp là tìm các sản phẩm tương đồng (Ví dụ tương đồng dựa trên domain hoặc kiểu âm nhạc tương đồng) và sau đó gán embedding bằng trung bình các embedding của các item tương đồng, để tạo ra vector khởi tạo ban đầu.
- **Embeddings and scalability:**
 - Sử dụng word2vec embedding Tốn tài nguyên tính toán, cần lượng lớn dữ liệu mới hiệu quả.
 - Ngoài ra, để gợi ý item, một việc cần làm là tính toán item gần nhất (cosine sim, nearest neighbor...). Điều này là vấn đề trong realtime. 1 cách xử lý là tính toán trước và lưu trữ K item gần nhất để dễ tìm kiếm sau này.
 - Một số dấu hiệu chỉ ra rằng phương pháp embedding có thể không hiệu quả như các mô hình tuần tự thuần túy như RNN, mặc dù điều này có

thể phụ thuộc vào tình huống s-ử dụng

Overall considerations

- Phiên được định nghĩa như thế nào? Các phiên nên được xác định từ lịch sử duyệt web, giao dịch hay cái gì? Một phiên nên kéo dài trong bao lâu? Điều gì xác định ranh giới của phiên (Ví dụ người dùng đang duyệt các mục trên web, các phiên có thể được phân định dựa trên sự không hoạt động của người dùng).
- Phần này sẽ nói về các thách thức của session-based recommender dựa trên các định nghĩa phiên khác nhau (thời lượng, thứ tự, ẩn danh, không ẩn danh...)

Vấn đề của phiên

- Ý định dài hạn hay ngắn hạn của người dùng quan trọng hơn?
 - Ta xem xét độ dài phiên, có thể chia làm ba loại:
 - Dài: > 10 tương tác
 - Trung bình: 4-9 tương tác
 - Ngắn: < 4 tương tác.
 - Dù nhiều tương tác có thể cung cấp nhiều thông tin nhưng chúng cũng có thể gây nhiễu do có khả năng chứa các tương tác ngẫu nhiên (do hành vi người dùng không chắc chắn) không liên quan đến tương tác khác → Hiệu suất kém. Ngược lại, các phiên ngắn lại chỉ mang thông tin hạn chế.
 - Các phiên trung bình thường tốt hơn trong việc nắm bắt đủ thông tin theo ngữ cảnh mà không bao gồm quá nhiều thông tin không liên quan.
- Các phiên có thể sắp xếp theo thứ tự hoặc không theo thứ tự. Nghĩa là các tương tác của người dùng có thể liên quan hoặc không liên quan đến nhau. Trong một số trường hợp, các phương pháp dựa trên co-visited có thể hoạt động tốt hơn các sequential model.
- Làm thế nào để học hỏi hiệu quả từ các hành động khác nhau của người dùng? (Phân biệt hoạt động duyệt web và hoạt động mua hàng?) Phiên có

chỉ nên chứa các mục đã được nhấp vào không? Hay nên có cả các hành động khác nhau?

- Vấn đề đánh giá